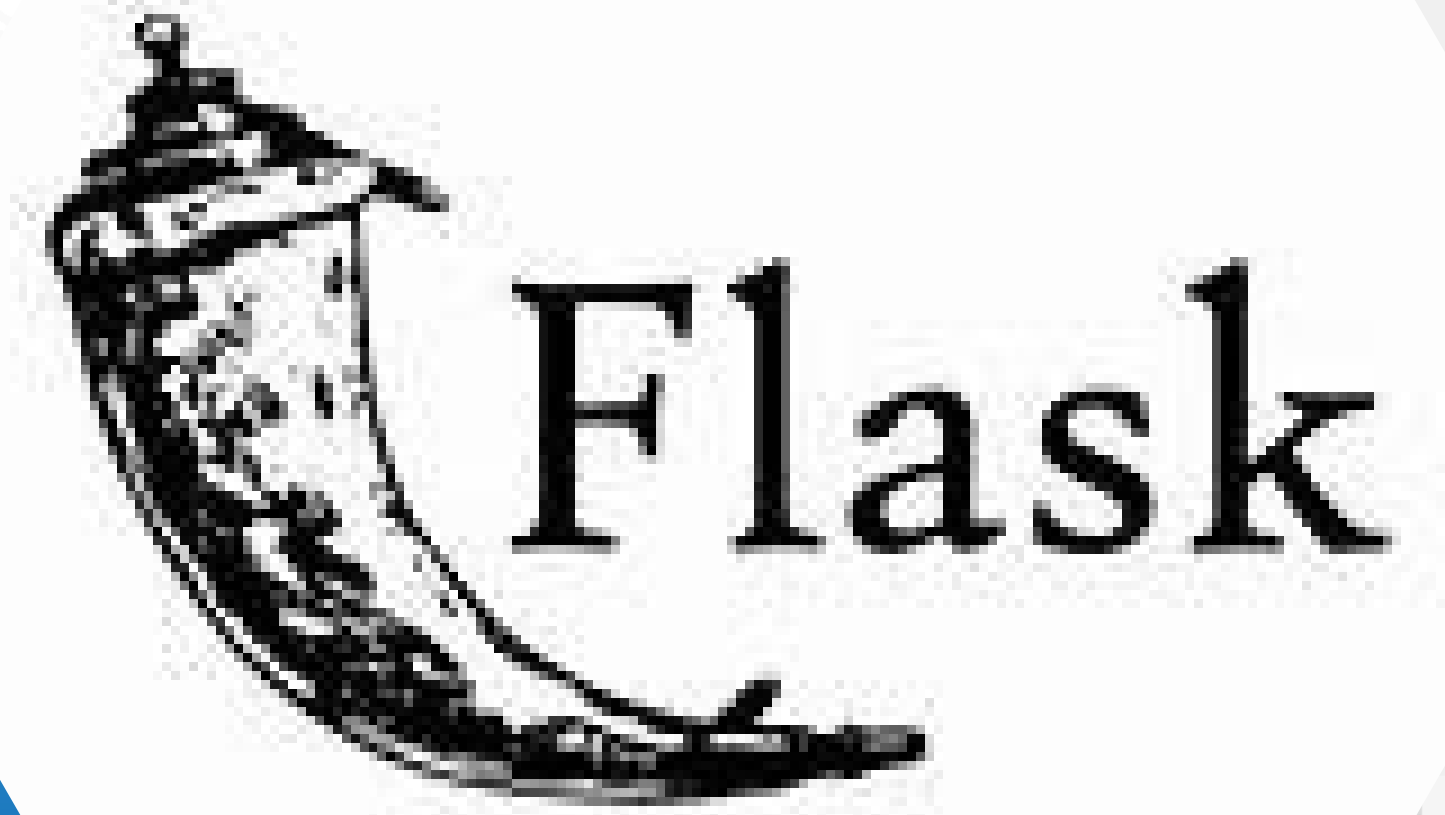


Introducción al desarrollo de APIs



EISHEL ALEXANDRA MARQUEZ AVALOS



¿Qué es Flask?

Flask es un microframework de Python.

Sirve para crear aplicaciones web de forma sencilla.

Es ligero, rápido y fácil de usar.

¿Que es un template en Flask?

Un template en Flask es un archivo HTML que contiene marcadores de posición para datos dinámicos, permitiendo generar páginas web personalizadas usando el motor de plantillas Jinja2.

Definición y función

En Flask, un template es un archivo que combina contenido estático (HTML, CSS, JavaScript) con marcadores de posición para datos dinámicos que se generan desde Python. Estos archivos se almacenan en el directorio templates de la aplicación y se utilizan para renderizar páginas web que se mostrarán en el navegador del usuario. La función principal de un template es separar la lógica de la aplicación de la presentación visual, facilitando el mantenimiento y la reutilización del código.

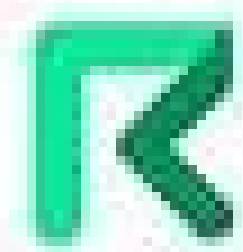
¿Cómo funciona Flask?

- Usa rutas (`@app.route`) para definir URLs
- Cada ruta ejecuta una función
- Puede devolver HTML o JSON
- Corre en un servidor local (localhost)

Código básico

- `from flask import Flask`
`app = Flask(name)`
- `@app.route('/')`
`def inicio():`
`return "Hola mundo"`
- `app.run(debug=True)`
- `debug=True` permite ver errores en tiempo real

¿Que es un request?



Request

Un request es una solicitud o petición que un cliente envía a un servidor a través de un modelo cliente-servidor. En el contexto de la tecnología de la información, especialmente en el protocolo HTTP, un request se refiere a un mensaje que indica a un servidor qué recurso o información se desea obtener.

¿Cuales son los tipos de request y su uso?

Los tipos de solicitudes HTTP son fundamentales para la comunicación entre clientes y servidores. Aquí hay un resumen de los principales tipos de solicitudes y su uso:

-Funcionamiento de:

- **GET:** Se utiliza para solicitar recursos específicos del servidor, como archivos o páginas web. No modifica el estado del recurso. Ejemplo: `GET /index.html` para obtener el contenido de una página web.
- **POST:** Se utiliza para enviar datos al servidor para crear o actualizar un recurso. Ejemplo: `POST /users` para agregar un nuevo usuario en una API.
- **PUT:** Se utiliza para actualizar un recurso existente o crear uno nuevo. Ejemplo: `PUT /users/1` para modificar los datos de un usuario existente.
- **DELETE:** Se utiliza para eliminar un recurso específico del servidor. Ejemplo: `DELETE /users/1` para borrar un usuario existente.

Swagger



- Herramienta para documentar APIs.
- Permite ver y probar endpoints.
- Facilita entender cómo funciona la API.

¿Como crear una Rest API con swagger?

- **swagger-jsdoc:** Genera el JSON de especificación OpenAPI a partir de comentarios JSDoc.
- **swagger-ui-express:** Muestra la documentación en una interfaz web interactiva.
- **Comentarios @swagger:** Definen cada endpoint, parámetros y respuestas.

Flask + Base de datos

- Se conecta a MySQL u otras bases.
- Permite guardar, consultar y modificar datos.

Flask + Swagger

- Se usa con librerías como Flasgger.
- Genera documentación automática.
- Permite probar la API sin código.